

## EXAMEN PARCIAL TEÓRICO - 2026/1

**Curso:** Estructuras de datos avanzadas (CS312)

**Grupo:** CCOMP6

**Profesores:** Erick Gómez Nieto  
Rosa Paccotacya Yanque

-----

### Pregunta 1 (Outcomes 1,2,6)(4 puntos):

- A. La función *ReInsert* del árbol  $R^*$  ( $R^*$  tree) es llamada únicamente una vez por cada nivel del árbol siempre que el nodo con *overflow* no sea la raíz.
- B. La función *PickSeeds* del árbol  $R$  ( $R$ -tree) tiene como objetivo encontrar cuáles son los mejores objetos/datos para dividir en dos un nodo con *overflow*.
- C. La variante del  $R$ -tree propuesta por Green (Green approach) propone introducir el cálculo del perímetro de las regiones para que estas sean cada vez más rectangulares, y así se evite la sobreposición.
- D. El BKD-tree es una estructura que reside únicamente en la memoria RAM para evitar los cuellos de botella de entrada y salida (I/O) del disco duro.
- E. En el algoritmo de inserción, los nuevos puntos se guardan primero en un buffer de memoria RAM antes de ser trasladados al disco.
- F. El pr-quadtrees tiene como estrategia la partición del espacio, tomando como centro de dicha partición la posición del punto que se está insertando.
- G. La maldición de la dimensionalidad se refiere al conjunto de fenómenos que aparece tras el crecimiento exponencial del espacio de los datos.
- H. Cuando un nodo debe dividirse durante la inserción, el algoritmo de división de Guttman (cuadrático) elige la división que minimiza la suma de los perímetros de los dos MBR resultantes.

## Pregunta 2 (Outcomes 1,2,6)(8 puntos):

Dado el siguiente código de RTree, implementa la función:

```
bool InsertRectRec(const Branch& a_branch, Node* a_node, Node**  
a_newNode, int a_level);
```

que realiza la inserción recursiva de una rama en el subárbol cuya raíz es a\_node, descendiendo hasta el nivel indicado por a\_level y propagando al nivel superior la información de un split si ocurre. En otras palabras, debes implementar la lógica de ChooseLeaf y de AdjustTree en esta función.

Parámetros:

- a\_branch: la entrada a insertar (rectángulo + dato si es hoja, o rectángulo + puntero a hijo si es interna).
- a\_node: nodo actual del descenso.
- a\_newNode: parámetro de salida. Si en este nivel hubo split, debe quedar apuntando al nodo hermano nuevo para que el llamador lo agregue al nivel superior.
- a\_level: nivel destino donde la rama debe quedar insertada.

Retorno:

- false si la inserción se completó en este nivel sin partir el nodo.
- true si el nodo se partió; en ese caso \*a\_newNode debe contener el hermano nuevo.

Ten en cuenta que este código se diferencia de Guttman en los siguientes puntos:

- ChooseLeaf: Guttman detiene el descenso cuando el nodo es hoja. Aquí la inserción puede ocurrir a un nivel arbitrario indicado por el parámetro a\_level, así que considera generalizar la lógica.
- El paso I4 del paper, crear una nueva raíz cuando la raíz se parte, NO se maneja en esta función; lo hace InsertRect (el llamador) cuando la llamada inicial a InsertRectRec (la que recibe la raíz como a\_node) retorna true

**El ejercicio tendrá una nota de 0 en caso de que el código entregado no compile.**

### **Pregunta 3 (Outcomes 1,2,6)(8 puntos):**

Se tiene el código adjunto del árbol R\*-tree ([rstartree.cpp](#)). Para que este pueda funcionar correctamente, implemente las siguientes funciones:

- `void Insert(const Entry& e, int level) // 1.5 puntos`
- `Node* OverflowTreatment(Node* N, int level) // 1.5 puntos`
- `ReInsert (4 puntos) // 5 puntos`